

CS149 Assignment 3

Yifan Luo

September 19, 2026

Contents

1	Part 1: CUDA Warm-Up 1: SAXPY	1
2	Part 2: CUDA Warm-Up 2: Parallel Prefix-Sum	2
3	Part 3: A Simple Circle Renderer	3
3.1	Challenges in Parallelizing Circle Blending	3
3.2	Our Renderer Design	4
3.3	Performance Results	5
3.4	How We Arrived at This Solution	6

Environment

All experiments in this assignment were conducted on my personal computer, which is equipped with an NVIDIA GeForce RTX 4060 Laptop GPU. The GPU has 24 streaming multiprocessors (SMs) and 8 GB of global memory. Since the experiments were performed on local hardware rather than the AWS `g5g.xlarge` instance provided by Stanford for CS149 students, the performance characteristics and experimental results reported here may differ from those obtained on the official course hardware.

Overview

In this write-up, we describe the three CUDA programs implemented in this assignment. The first two programs, SAXPY and exclusive scan, serve as introductory exercises for practicing fundamental CUDA programming concepts. Our primary effort is devoted to the third program: a parallel CUDA renderer for drawing semi-transparent colored circles.

1 Part 1: CUDA Warm-Up 1: SAXPY

In this part, we implemented a SAXPY routine in CUDA and measured both the execution time of the CUDA kernel itself and the end-to-end execution time, which includes transferring data from the CPU to the GPU, executing the kernel, and transferring the results back from the GPU to the CPU. The results are shown below:

Running 3 timing tests:

Effective BW by CUDA kernel: 6.715 ms	[166.424 GB/s]
Effective BW by CUDA saxpy: 158.066 ms	[7.070 GB/s]
Effective BW by CUDA kernel: 5.284 ms	[211.512 GB/s]
Effective BW by CUDA saxpy: 155.748 ms	[7.176 GB/s]
Effective BW by CUDA kernel: 5.270 ms	[212.057 GB/s]
Effective BW by CUDA saxpy: 142.631 ms	[7.835 GB/s]

For comparison, the serial SAXPY implementation from Assignment 1 achieved the following performance:

[saxpy serial]:	[11.446] ms	[26.037] GB/s	[3.495] GFLOPS
-----------------	-------------	---------------	----------------

The workloads used in the two assignments are different, so directly comparing their execution times would not be meaningful. Instead, it is more appropriate to compare the achieved memory throughput. Note, however, that the CPU implementation may incur an additional memory read due to the read-for-ownership (RFO) mechanism. Therefore, even if the reported throughput were the same (which is not the case here), the actual amount of useful work performed would still differ.

We can see that the throughput of the GPU kernel is substantially higher than that of the serial CPU implementation. However, once the cost of transferring data between the CPU and GPU is included, the end-to-end CUDA implementation actually achieves a lower effective throughput. This is because the bandwidth of GPU device memory is much higher than the bandwidth of the PCIe interconnect between the CPU and GPU.

The measured values can also be compared with the theoretical bandwidth limits of the hardware. The theoretical device-memory bandwidth of the GPU on my machine is approximately 256 GB/s, while the highest observed kernel throughput is about 212 GB/s. This is reasonably close to the theoretical limit and is consistent with SAXPY being a memory-bound workload.

In contrast, the theoretical PCIe bandwidth of my machine is approximately 15.75 GB/s, whereas the highest observed end-to-end throughput is only about 7.8 GB/s. This gap is expected because the end-to-end measurement includes not only PCIe transfers but also additional overheads associated with host memory and data movement. In particular, transfers from pageable host memory may require an intermediate copy through pinned memory before DMA transfer to the GPU, preventing the application from fully utilizing the theoretical PCIe bandwidth.

2 Part 2: CUDA Warm-Up 2: Parallel Prefix-Sum

In this part, we implemented a parallel exclusive scan routine in CUDA and then used it to implement `find_repeats`. Given an integer array `A`, `find_repeats` returns the indices `i` such that `A[i] == A[i+1]`.

Our parallel scan implementation follows the Blelloch scan algorithm. To implement `find_repeats`, we first construct an auxiliary array `flags`, where `flags[i] = 1` if `A[i] == A[i+1]`, and `flags[i] = 0` otherwise. We then perform an exclusive scan on `flags` to obtain an array `offsets`. Finally, for every index `i` such that `flags[i] == 1`, we write `i` to `list[offsets[i]]`.

The performance results are shown below:

Scan Score Table:

Element Count	Ref Time	Student Time	Score
1000000	1.677	0.508	1.25
10000000	10.281	5.837	1.25
20000000	19.262	13.247	1.25
40000000	36.556	23.413	1.25
		Total score:	5.0/5.0

Find_repeats Score Table:

Element Count	Ref Time	Student Time	Score
1000000	2.195	1.311	1.25
10000000	16.131	11.046	1.25
20000000	31.219	20.768	1.25
40000000	55.7	44.121	1.25
		Total score:	5.0/5.0

Our implementation outperforms the reference implementation on all tested input sizes for both the exclusive scan and `find_repeats`.

3 Part 3: A Simple Circle Renderer

In this part, we implemented a simple circle renderer in CUDA. The renderer draws semi-transparent colored circles and blends them with the existing pixel colors. A correct implementation produces results such as those shown below.

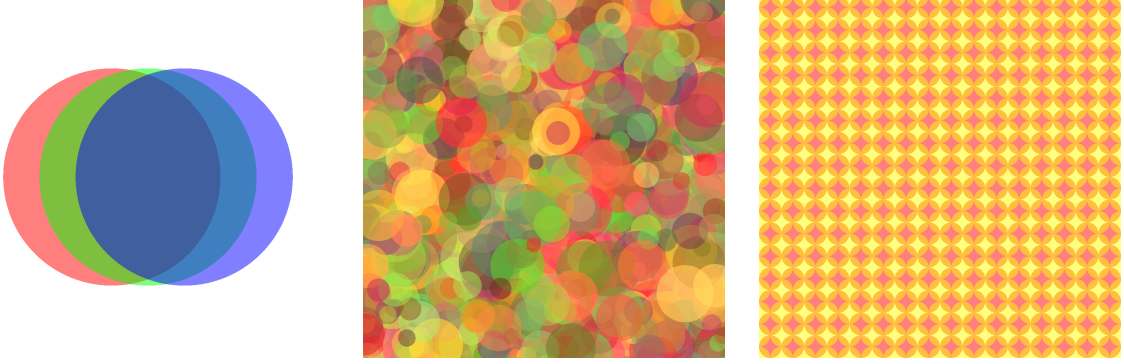


Figure 1: Renderer outputs for `rgb` (left), `rand100k` (middle), and `pattern` (right).

3.1 Challenges in Parallelizing Circle Blending

The main challenge in this part is that blending circles onto pixels is not trivially parallelizable.

The blending operation can be written as follows:

```

1 result_r = C_alpha * C_r + (1.0 - C_alpha) * P_r
2 result_g = C_alpha * C_g + (1.0 - C_alpha) * P_g
3 result_b = C_alpha * C_b + (1.0 - C_alpha) * P_b

```

where `C_alpha` denotes the opacity of the circle, and `(P_r, P_g, P_b)` and `(C_r, C_g, C_b)` denote the colors of the current pixel and the circle, respectively.

Importantly, this blending operation is not commutative: blending circle A over circle B generally produces a different result from blending circle B over circle A. Therefore, the relative order in which overlapping circles are blended must be preserved.

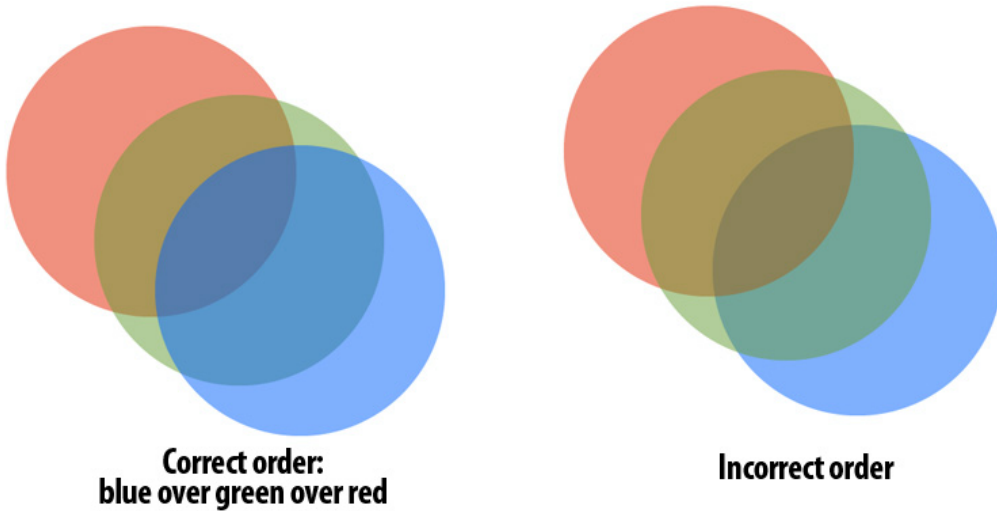


Figure 2: The blending order of overlapping circles affects the final pixel color.

This immediately rules out a naive parallelization strategy in which each circle is assigned to an independent CUDA thread. CUDA does not guarantee the execution order of threads, so such an implementation cannot reliably preserve the required circle ordering. In addition, when multiple circles overlap the same pixel, different threads may concurrently read and write that pixel, resulting in a data race.

3.2 Our Renderer Design

To overcome the challenges described above, we adopt a different work-assignment strategy from the naive design. Instead of assigning one CUDA thread to each circle, we assign one CUDA thread to each pixel. This immediately eliminates the data race on pixel values, since each pixel is read and written by only one thread. To preserve the required blending order, we construct, for each region of the image, an ordered list of circles that may intersect that region. Each pixel then processes the circles in the corresponding list in order.

We first divide the image into 16×16 tiles.

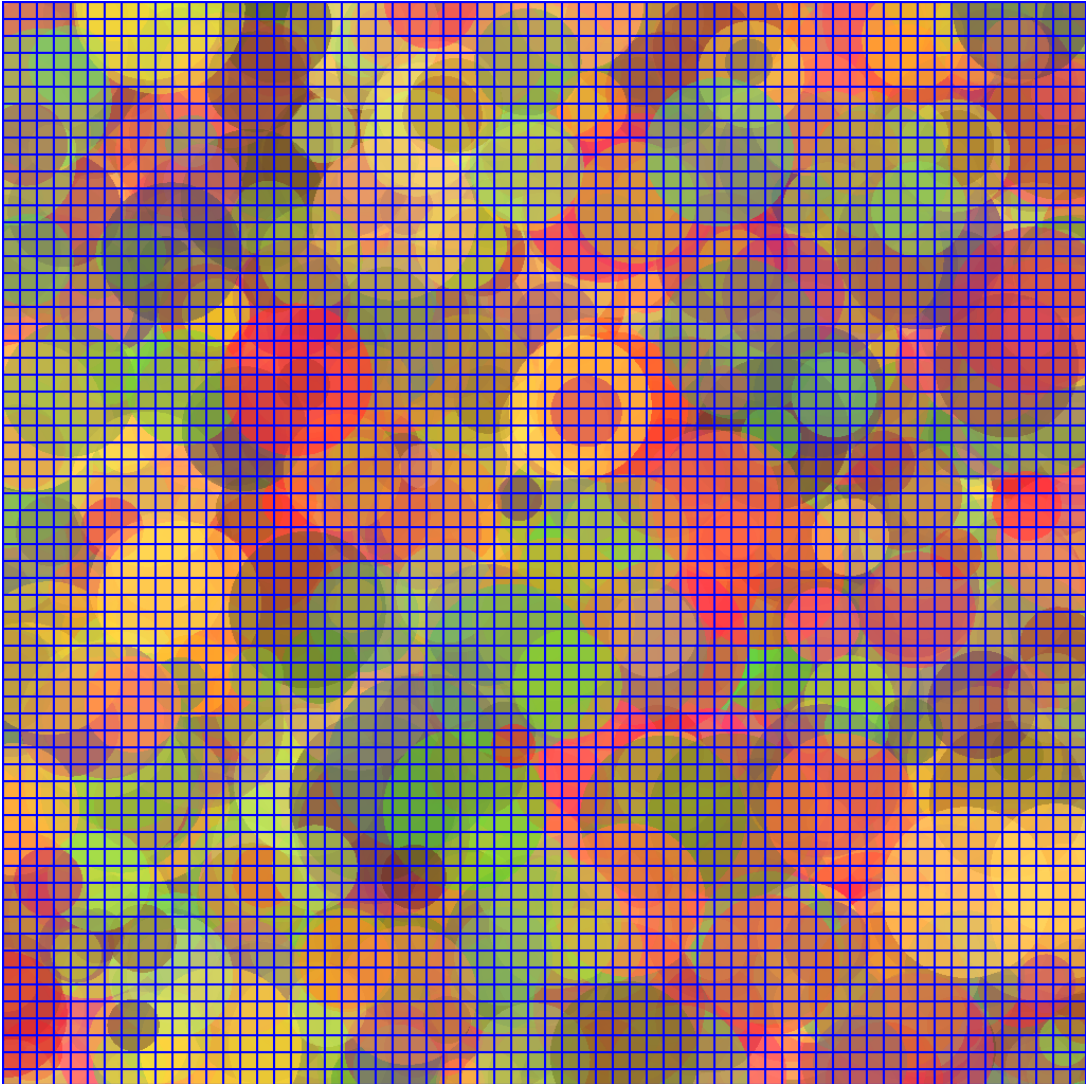


Figure 3: Tiling of the `rand100k` scene.

A tile is the granularity at which we maintain a circle list. If a circle intersects a tile, we conservatively treat the circle as potentially relevant to every pixel in that tile. When processing a circle, the blending function checks whether the individual pixel actually lies within the circle; if not, it simply leaves the pixel unchanged.

Our final implementation consists of two stages. In the first stage, we construct an ordered circle list

for each tile. In the second stage, one CUDA thread is assigned to each pixel, and each thread blends the circles in its tile’s corresponding list in the required order.

Stage 1. In Stage 1, we construct the circle lists for all tiles.

- **Step 1: Count tile intersections.** We assign one CUDA thread to each circle, using 256 threads per block. Each thread counts the number of tiles intersected by its circle and stores the result in `devTileCount`.
- **Step 2: Compute write offsets.** We perform an scan on `devTileCount` to obtain `devCountOffsets`. These offsets assign each circle a disjoint range in the flattened tile-list array, allowing the threads to write their results without synchronization.
- **Step 3: Materialize circle–tile pairs.** We again assign one CUDA thread to each circle. Each thread enumerates the tiles intersected by its circle and writes the corresponding tile indices into `devTileLists`. At the same positions, the corresponding circle indices are written into `devCircleLists`.
- **Step 4: Group circles by tile.** We perform a stable sort on the pairs (`devTileLists`, `devCircleLists`), using `devTileLists` as the sorting key. After sorting, all circles associated with the same tile become contiguous in `devCircleLists`. Because the sort is stable, the relative order of circles within each tile is preserved.
- **Step 5: Determine segment boundaries.** Finally, in addition to `devCircleLists` itself, we record the beginning and end of the circle list associated with each tile. These segment boundaries can be obtained directly from the sorted `devTileLists`.

Stage 2. In Stage 2, each pixel is blended using the `devCircleLists` associated with its tile.

- **Step 1: Load circle data into shared memory.** We assign one CUDA thread to each pixel, with a 16×16 thread block corresponding exactly to one image tile. Before blending, the circle data required by the block are loaded into `__shared__` memory. This is beneficial because all pixels within a tile use the same circle list, resulting in substantial data reuse within the block. Each thread loads the radius, color, and position of one circle from the tile’s circle list. Since a block contains only 256 threads, long circle lists are processed in chunks of at most 256 circles.
- **Step 2: Blend the pixels.** After one chunk of circle data has been loaded into shared memory, each thread processes its assigned pixel by traversing the chunk in order and blending each relevant circle onto the pixel. If a pixel does not lie inside a particular circle, the blending routine simply leaves that pixel unchanged. The block then proceeds to the next chunk until the entire circle list for the tile has been processed.

3.3 Performance Results

Our renderer achieves strong performance across the benchmark scenes and outperforms the reference implementation in many cases, particularly for scenes with heavier workloads. For lighter scenes, the fixed overhead of our rendering pipeline becomes more significant, so the performance is closer to that of the reference implementation.

Score table:

Scene Name	Ref Time (T_ref)	Your Time (T)	Score
rgb	0.5357	0.5149	9
rand10k	3.1516	3.4415	9
rand100k	27.0007	10.0063	9
pattern	0.8666	0.9363	9
snowsingle	17.4707	2.41	9

biglitttle	15.6361	13.2834	9	
rand1M	161.3823	11.6116	9	
micro2M	311.1123	6.6967	9	

		Total score:	72/72	

Our implementation is slightly slower than the reference on **rand10k** and **pattern**, by approximately 9% and 8%, respectively. In contrast, it provides substantial speedups on the more computationally intensive scenes. For example, it achieves approximately a 13.9 $\times$ speedup on **rand1M** and a 46.5 $\times$ speedup on **micro2M**. These results suggest that the additional preprocessing overhead of our pipeline is amortized effectively as the workload increases.

3.4 How We Arrived at This Solution

In this section, we briefly describe the evolution of our renderer design and the optimizations that led to the final implementation.

In the first version of our renderer, Stage 1 was substantially different from the current design. We explicitly constructed a large array containing all tile-circle pairs, with each entry storing a flag indicating whether the corresponding tile and circle intersected. We then compacted this sparse representation into a dense list of intersecting circles. This approach requires

$$\mathcal{O}(N_{\text{tiles}}N_{\text{circles}})$$

space, which quickly became impractical. In particular, the **rand1M** scene caused an out-of-memory error. Since only the dense list is ultimately needed, we redesigned Stage 1 to first count the number of tile intersections for each circle and then materialize only the actual intersections. This avoids constructing the full sparse tile-circle matrix.

Another important design choice in Stage 1 is that we exploit circle-level parallelism rather than tile-level parallelism. An alternative implementation could assign work by tile, directly count the intersecting circles for each tile, and construct the corresponding lists without a stable sort. However, in most benchmark scenes, the number of circles is much larger than the number of tiles. Exploiting circle-level parallelism therefore exposes substantially more parallel work. We believe this is one of the reasons our implementation performs particularly well on scenes such as **rand1M** and **micro2M**, which contain millions of circles.

After this redesign, the renderer already achieved good overall performance, but it still failed to outperform the reference implementation on several scenes. These included **biglitttle** and several relatively small workloads, namely **rgb**, **rand10k**, and **pattern**.

For **biglitttle**, profiling showed that the main bottleneck was **kernelBlendPixelByStartsEnds**. We therefore focused on improving its memory-access behavior. In the original version of **shadePixel**, which is called by **kernelBlendPixelByStartsEnds**, the pixel value was accessed in global memory during every iteration over the circle list. We changed this implementation to keep the pixel value in a local accumulator throughout the traversal and perform only a single global-memory write at the end.

The second optimization was the use of `__shared__` memory, as described in Section 3.2. Since all threads in a block process pixels from the same tile, they repeatedly access the same circle data. Loading this data into shared memory allows the threads to reuse it without repeatedly fetching it from global memory. This optimization introduces synchronization overhead: all 256 threads must finish loading a chunk before blending begins, and they must finish processing that chunk before the shared-memory buffer can be reused for the next one. In practice, however, the synchronization cost is much smaller than the benefit obtained from reducing global-memory traffic. After applying these two optimizations, our implementation was able to outperform the reference implementation on **biglitttle**.

For the smaller workloads, profiling revealed a different bottleneck: the first call to a Thrust primitive inside **render**. In our implementation, the first such call was **thrust::inclusive_scan**, which appeared to take roughly 1 ms even for extremely small inputs. For example, in **rgb**, the scan operates on an array of only three elements, so the scan computation itself clearly cannot account for such a large execution time.

To investigate this behavior, we replaced **thrust::inclusive_scan** with an equivalent CUB scan. After this change, the apparent bottleneck immediately shifted to **thrust::stable_sort_by_key**. This suggested that the additional latency was associated with the first invocation of Thrust rather than with the scan operation itself.

We therefore introduced a warm-up call in `setup`, where Thrust is used to sort an array of 1024 zero-valued elements. After this warm-up, the measured execution time of `render` decreased by approximately 1 ms, which was sufficient for our implementation to become competitive with the reference implementation on the smaller benchmark scenes.

It is worth emphasizing that this last modification is better viewed as a benchmark-oriented workaround than as a fundamental algorithmic optimization. A cleaner solution would be to avoid Thrust for sufficiently small workloads altogether. For example, when `numCircles` falls below a chosen threshold, we could use specialized scan and sorting routines with lower launch and initialization overhead, while retaining Thrust for larger inputs.